CalisFacu v1.0

Plan de Aseguramiento de Calidad de Software (SQAP)

Conforme a IEEE Std 730-2002, agrupado según las áreas de IEEE Std 730-2014

Equipo L-001 · COM450 Calidad de Software · Gestión II/2026 · Versión del plan 0.2

1. Propósito

Este plan cubre el desarrollo de **CalisFacu v1.0**, sistema web de registro de calificaciones para docentes (Laravel + MySQL 8). Abarca las fases de requerimientos, diseño, construcción y pruebas de los casos de uso principales: **Registrar calificación**, **Calcular promedios** y **Exportar historial a PDF**. El sistema contempla dos roles: Docente y Administrador. El plan rige durante la Gestión II/2026 y lo ejecuta el equipo L-001.

El sistema se clasifica en el **nivel de integridad 2 (Moderado)** según el esquema de IEEE Std 1012. Justificación: una falla (por ejemplo, un error en el cálculo de promedios o la indisponibilidad al exportar el PDF) causa una inconveniencia operativa recuperable mediante recálculo o corrección en la base de datos, pero no provoca pérdida de vidas, daño ambiental ni pérdida económica mayor. Dentro del sistema, los módulos **Registro de calificación** y **Cálculo de promedios** se consideran **críticos**, porque modifican y calculan notas oficiales.

2. Documentos de referencia

Código	Documento	Versión	Ubicación
SRS	Especificación de requerimientos de CalisFacu (extracto de RNF en el Anexo)	1.0	docs/srs.pdf
SAD	Documento de arquitectura: Laravel + MySQL 8	1.0	docs/sad.pdf
IEEE 730	IEEE Std 730-2002, Software Quality Assurance Plans	2002	IEEE Xplore (biblioteca USFX)
ISO 25010	ISO/IEC 25010, modelo de calidad del producto	2011	Biblioteca USFX
IEEE 1012	IEEE Std 1012, niveles de integridad (Sección 1)	2016	IEEE Xplore (biblioteca USFX)
IEEE 1028	IEEE Std 1028, revisiones y auditorías (Sección 6)	2008	IEEE Xplore (biblioteca USFX)

Código	Documento	Versión	Ubicación
PSR-12	PHP Standards Recommendation 12: estilo de código PHP	1.0	php-fig.org/psr/psr-12

3. Gestión

El equipo L-001 tiene dos integrantes, por lo que cada uno asume más de un rol: **Lujan** es Líder de proyecto, Arquitecto y Desarrollador backend; **Galvan** es Responsable SQA y Desarrollador frontend / Tester. **Regla de independencia:** quien escribe un artefacto no lo revisa ni lo aprueba. Galvan no programa los módulos críticos (Registro de calificación y Cálculo de promedios), que escribe Lujan; por eso Galvan aprueba sus pull requests. El código de Galvan (interfaz y scripts de Cypress y JMeter) lo revisa Lujan. El Responsable SQA tiene autoridad para rechazar cualquier entregable que no cumpla los objetivos de la Sección 5.

Rol	Responsabilidad principal	Integrante
Líder de proyecto	Coordina el equipo, vigila el cronograma, fija la prioridad de los defectos y rinde cuentas por el proyecto.	Lujan
Arquitecto	Define la estructura en Laravel y el diseño de la base de datos MySQL.	Lujan
Responsable SQA	Mantiene el SQAP, aprueba los pull requests de los módulos críticos y audita el cumplimiento del plan. No programa módulos críticos.	Galvan
Desarrollador frontend / Tester	Construye la interfaz y ejecuta las pruebas E2E (Cypress) y de carga (JMeter).	Galvan
Desarrollador backend	Desarrolla la lógica de negocio en Laravel (módulos críticos y Exportar PDF), la base de datos y las pruebas PHPUnit.	Lujan

Independencia parcial y cómo se compensa: como el Responsable SQA también es Tester, las evidencias de prueba que él produce (informes de Cypress y JMeter) no las audita él mismo: las audita el Líder en la auditoría de cierre (revisión cruzada). En la inspección formal del SAD participa como inspector un integrante de otro equipo del curso (Sección 6).

Matriz RACI (R ejecuta, A rinde cuentas, C es consultado, I es informado; exactamente una A por fila). Lujan cubre las columnas Líder, Arquitecto y Dev; Galvan cubre SQA y Tester.

Actividad	Líder	Arquitecto	SQA	Dev	Tester
Redactar y mantener el SQAP	A	C	R	I	C
Revisar el SRS (autor: Líder)	C	C	A	I	R
Revisar código de módulos críticos (autor: Dev)	I	I	A/R	C	I
Revisar código de interfaz y scripts de prueba (autor: Tester)	I	A	I	R	C
Ejecutar pruebas de carga	I	A	C	I	R
Aprobar una release	A	C	R	I	C
Auditar el cumplimiento del plan	R	I	A/R	I	C

4. Documentación

Documento	Responsable	Fecha prevista	Criterio de adecuación	Tipo de revisión
SRS	Líder (Lujan)	Semana 1	Cada historia de usuario tiene criterios de aceptación y cada RNF tiene métrica y valor verificable.	Inspección con el cliente (docentes) y el Responsable SQA.
SAD	Arquitecto (Lujan)	Semana 2	Detalla el esquema MySQL, los patrones de Laravel y las decisiones que cubren RNF-03 y RNF-04.	Inspección Fagan (Sección 6).
Manual de usuario	Tester (Galvan)	Semana 7	Cubre los tres casos de uso con capturas; un docente piloto completa "Registrar calificación" usando solo el manual.	Walkthrough con un docente piloto; revisa Lujan.
Informe de pruebas	Tester (Galvan)	Cierre de cada sprint (semanas 3, 5 y 7)	Incluye los resultados de PHPUnit, Cypress y JMeter contra los objetivos M01 a M09 de la Sección 5.	Auditoría de cierre; la audita el Líder porque el autor es Galvan.

5. Estándares, prácticas, convenciones y métricas

Estándar de codificación: Se adopta de manera estricta el estándar oficial PSR-12 para el lenguaje PHP. **Justificación:** Es la convención nativa recomendada por el framework Laravel, lo que garantiza la legibilidad del código entre los distintos desarrolladores del equipo y asegura la compatibilidad total con el analizador estático SonarQube para la medición de deuda técnica.

Convenciones de nombres:

- **Clases y Modelos:** PascalCase (ejemplo: RegistroCalificacion).
- **Variables y Métodos:** camelCase (ejemplo: calcularPromedio()).
- **Tablas de base de datos:** snake_case en plural (ejemplo: calificaciones_estudiantes).
- **Endpoints (Rutas API):** kebab-case en minúsculas (ejemplo: /api/exportar-historial-pdf).
- **Pruebas (Tests):** snake_case con el prefijo *test* (ejemplo: test_calculo_promedio_correcto).

Métricas (detalle en la hoja Métricas de `matrices-sqap.xlsx`):

Característica 25010	Métrica y fórmula	Instrumento, condiciones y momento	Objetivo	RNF / riesgo	Responsable
Adecuación funcional (corrección)	M01 · % de pruebas unitarias de Cálculo de promedios que pasan = pruebas en verde / pruebas totales	PHPUnit (P1), suite completa; en cada build de Jenkins	100 %	RNF-01	Desarrollador backend (Lujan)
Adecuación funcional (completitud)	M02 · % de escenarios E2E de Registrar calificación y Exportar PDF que pasan = escenarios en verde / totales	Cypress (P4), staging con datos sintéticos; en cada release candidate	100 %	RNF-02	Tester (Galvan)
Eficiencia de desempeño (comportamiento temporal)	M03 · p95 del tiempo de respuesta de Exportar historial a PDF = percentil 95 de los tiempos medidos	JMeter (P5), 100 usuarios concurrentes, 10 min, staging; en cada release candidate	≤ 2000 ms	RNF-03	Tester (Galvan)

Característica 25010	Métrica y fórmula	Instrumento, condiciones y momento	Objetivo	RNF / riesgo	Responsable
Eficiencia de desempeño (capacidad)	M04 · Tasa de error al registrar notas bajo carga = peticiones fallidas / peticiones totales	JMeter (P5), 100 usuarios concurrentes, 10 min, staging; en cada release candidate	< 1 %	R05	Tester (Galvan)
Mantenibilidad (capacidad de prueba)	M05 · Cobertura de líneas del código nuevo = líneas nuevas cubiertas / líneas nuevas	PHPUnit + PCOV, reportado en SonarQube (P3); en cada pull request	≥ 80 %	R03	Responsable SQA (Galvan)
Mantenibilidad (capacidad de prueba)	M06 · Cobertura de líneas del módulo crítico Cálculo de promedios = líneas cubiertas / líneas del módulo	PHPUnit + PCOV (P1, P6), filtro app/Services/Promedios; en cada build	≥ 90 %	RNF-01	Responsable SQA (Galvan)
Mantenibilidad (modificabilidad)	M07 · Líneas duplicadas en código nuevo = líneas duplicadas / líneas nuevas	SonarQube (P3); en cada pull request	≤ 3 %	RNF-05	Responsable SQA (Galvan)
Seguridad (confidencialidad)	M08 · Vulnerabilidades conocidas = conteo en código nuevo (SonarQube) y en dependencias (composer audit)	SonarQube (P3) y composer audit; en cada pull request	0	RNF-04	Responsable SQA (Galvan)
Seguridad (integridad)	M09 · % de rutas de escritura de calificaciones que rechazan (HTTP 403) a un usuario sin rol Docente = rutas que rechazan / rutas de escritura	Feature tests de PHPUnit (P1); en cada build	100 %	RNF-04	Desarrollador backend (Lujan)

El mismo número en dos lugares. Cada umbral de la tabla anterior está configurado en la herramienta que lo hace cumplir. Si un número cambia, se cambia en esta sección, en matrices-sqap.xlsx y en la configuración, en el mismo pull request (Sección 16).

Métrica del plan	Dónde se hace cumplir	Configuración
M01 · 100 % de pruebas en verde	Jenkinsfile, etapa Pruebas	phpunit --log-junit + paso junit: el build falla si una prueba falla
M02 · 100 % de escenarios E2E	Jenkinsfile, etapa E2E	npx cypress run: código de salida distinto de 0 si falla un escenario
M03 · p95 ≤ 2000 ms	Jenkinsfile, etapa Rendimiento	statistics.json de JMeter: jq -e ".Total.pct2ResTime <= 2000"
M04 · tasa de error < 1 %	Jenkinsfile, etapa Rendimiento	statistics.json de JMeter: jq -e ".Total.errorPct < 1"
M05 · cobertura del código nuevo ≥ 80 %	Quality Gate "CalisFacu" de SonarQube	Coverage on New Code is less than 80.0
M06 · cobertura de Cálculo de promedios ≥ 90 %	Jenkinsfile, etapa Cobertura	coverage-check build/promedios.xml 90
M07 · duplicación ≤ 3 %	Quality Gate "CalisFacu" de SonarQube	Duplicated Lines (%) on New Code is greater than 3.0
M08 · 0 vulnerabilidades	Quality Gate + Jenkinsfile, etapa Dependencias	Security Rating on New Code is worse than A; composer audit
M09 · 100 % de rutas protegidas	Jenkinsfile, etapa Pruebas	Suite Autorizacion de PHPUnit (corre junto con M01)

Fragmentos de configuración (Evidencia 4):

```
// Jenkinsfile (fragmento): umbrales de la Sección 5
stage('Pruebas') {                    // M01, M09
  steps {
    sh '''vendor/bin/phpunit --log-junit build/junit.xml \
          --coverage-clover build/clover.xml'''
  }
  post { always { junit 'build/junit.xml' } }
}
stage('Cobertura') {                  // M06: módulo crítico >= 90 %
  steps {
```

```
    sh '''vendor/bin/phpunit --testsuite Promedios \
          --coverage-filter app/Services/Promedios \
          --coverage-clover build/promedios.xml'''
    sh 'vendor/bin/coverage-check build/promedios.xml 90'
  }
}
stage('Análisis estático') {          // M05, M07, M08
  steps {
    withSonarQubeEnv('sonarqube') {
      sh 'sonar-scanner -Dsonar.php.coverage.reportPaths=build/clover.xml'
    }
    timeout(time: 10, unit: 'MINUTES') {
      waitForQualityGate abortPipeline: true
    }
  }
}
stage('Dependencias') {               // M08
  steps { sh 'composer audit' }
}
stage('E2E') {                        // M02, solo release candidate
  when { branch 'release/*' }
  steps { sh 'npx cypress run --config baseUrl=$STAGING_URL' }
}
stage('Rendimiento') {                // M03 y M04, solo release candidate
  when { branch 'release/*' }
  steps {
    sh '''jmeter -n -t perf/exportar-pdf.jmx -Jusuarios=100 \
          -Jduracion=600 -l perf/pdf.jtl -e -o perf/pdf'''
    sh '''jmeter -n -t perf/registrar-notas.jmx -Jusuarios=100 \
          -Jduracion=600 -l perf/notas.jtl -e -o perf/notas'''
    // M03: p95 <= 2000 ms            M04: tasa de error < 1 %
    sh 'jq -e ".Total.pct2ResTime <= 2000" perf/pdf/statistics.json'
    sh 'jq -e ".Total.errorPct < 1" perf/notas/statistics.json'
  }
  post { always { archiveArtifacts 'perf/**' } }
}
```

```
Quality Gate "CalisFacu" en SonarQube:
Coverage on New Code is less than 80.0                -> falla   (M05)
Duplicated Lines (%) on New Code is greater than 3.0  -> falla   (M07)
Security Rating on New Code is worse than A           -> falla   (M08)
```

El percentil 95 corresponde a pct2ResTime porque el dashboard de JMeter usa los percentiles 90, 95 y 99 por defecto; jq -e termina con error (y el build falla) si la condición es falsa. Los planes .jmx leen la carga con ${__P(usuarios)} y ${__P(duracion)}.

6. Revisiones de software

Revisión	Artefacto	Cuándo	Quién revisa	Lista de verificación	Criterio de aceptación
Revisión de requerimientos	SRS	Semana 1	Cliente (docentes) y Responsable SQA (Galvan). El autor (Lujan) presenta, pero no aprueba.	Cada historia con criterios de aceptación; cada RNF con métrica y valor; reglas de cálculo de promedios con casos de ejemplo.	Acta de aceptación del SRS firmada por el cliente; 0 RNF sin criterio verificable.
Inspección Fagan del SAD	Documento de arquitectura	Semana 2	Moderador y escriba: Galvan (SQA). Lector e inspector: un integrante de otro equipo del curso. Autor: Lujan (responde, no inspecciona).	Esquema MySQL normalizado; índices para consultas de notas y PDF; patrones de Laravel; PSR-12 planificado.	0 defectos mayores abiertos; acta de inspección en docs/actas/.
Revisión de código por PR	Código nuevo (Laravel e interfaz)	En cada pull request a main	Módulos críticos y backend: Galvan (SQA). Interfaz y scripts de prueba: Lujan. Nadie aprueba su propio PR.	Quality Gate en verde; etapas Pruebas y Cobertura en verde; PSR-12; pruebas nuevas para el cambio.	PR aprobado en GitHub por el revisor asignado (main protegida).
Auditoría de cierre	Informe de pruebas, evidencias y SQAP	Cierre de cada sprint y antes de cada release	Galvan audita los artefactos de desarrollo; Lujan audita las evidencias de prueba que produjo Galvan.	Siete indicadores de la regla del plan verificable; objetivos M01 a M09 cumplidos con evidencia (Sección 12).	Acta de auditoría firmada por ambos; release aprobada según la RACI.

7. Pruebas

Nivel	Técnica y herramienta	Responsable	Criterio de entrada	Criterio de salida
Unitario	TDD obligatorio en Registro de calificación y Cálculo de promedios; PHPUnit + PCOV	Desarrollador backend (Lujan)	Historia con criterios de aceptación	100 % de pruebas en verde (M01); cobertura ≥ 90 % en Cálculo de promedios (M06) y ≥ 80 % en código nuevo (M05)
Integración	Feature tests de Laravel (PHPUnit) contra MySQL 8 de prueba en un contenedor Docker, la misma versión que producción	Desarrollador backend (Lujan)	Unitarias en verde	100 % de feature tests en verde; 100 % de rutas de escritura protegidas por rol (M09)
Sistema	E2E con Cypress (M02); carga con JMeter (M03, M04)	Tester (Galvan)	Build sano desplegado en staging	100 % de escenarios E2E en verde; p95 de Exportar PDF ≤ 2000 ms; tasa de error < 1 %
Aceptación	Escenarios con docentes del cliente sobre los tres casos de uso	Líder (Lujan) con el cliente	Pruebas de sistema aprobadas	0 defectos S1 o S2 abiertos; acta de aceptación firmada por el cliente

Cobertura mínima global y por módulo crítico:

Módulo crítico Cálculo de promedios (app/Services/Promedios): ≥ 90 % de líneas (M06).

Código nuevo de todo el sistema, medido por SonarQube: ≥ 80 % de líneas (M05).

La integración usa MySQL y no SQLite en memoria, porque las diferencias de SQL entre ambos motores (tipos, collation, funciones de fecha) dejarían pasar defectos que solo aparecen en producción.

8. Reporte de problemas y acción correctiva

Estados del defecto y transiciones:

Todo defecto sigue este ciclo: **Nuevo** (lo registra el Tester o un usuario) → **Asignado** (el Líder fija la prioridad y lo asigna) → **En progreso** (Desarrollador) → **Resuelto** (Desarrollador) → **Verificado** (Tester, si la prueba de confirmación pasa) → **Cerrado** (Responsable SQA).

Si la prueba de confirmación falla, el defecto pasa de Resuelto a **Reabierto**; un defecto Cerrado que reaparece también pasa a Reabierto. Desde Reabierto, el Líder lo vuelve a Asignado.

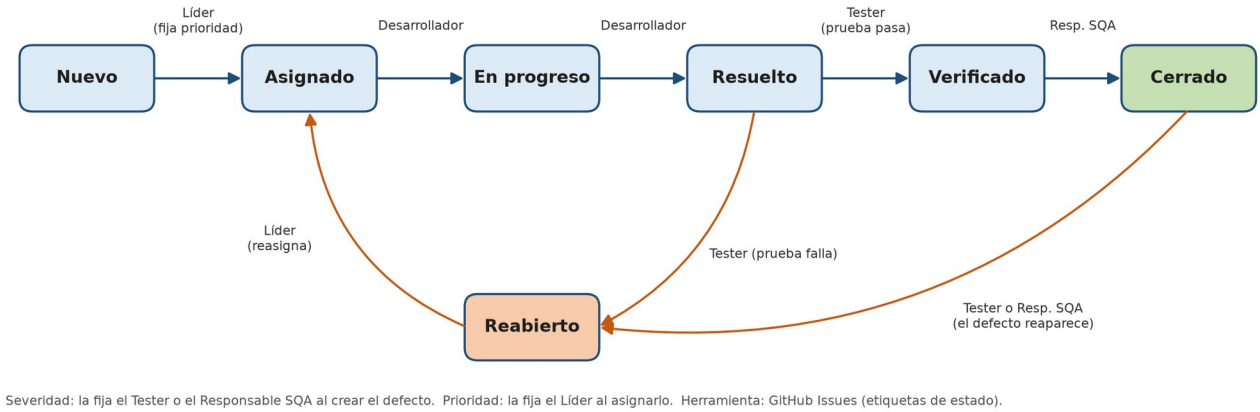


Figura 1. Estados del defecto y quién mueve cada transición.

Diferencia entre severidad y prioridad:

- **Severidad:** Indica el nivel de daño o impacto que el defecto causa en el sistema. La define el Tester o Responsable SQA.
- **Prioridad:** Indica el orden o la urgencia con la que el defecto debe ser corregido respecto a otras tareas. La define el Líder de Proyecto según el negocio.

Herramienta de gestión: GitHub Issues, con la plantilla plantilla-reporte-defecto.md y una etiqueta por estado. Ejemplo: CAL-001 (Anexo de la entrega).

SLA de resolución de defectos. Los tiempos se cuentan en horario hábil (lunes a viernes, 8:00 a 18:00). El equipo no tiene guardia de fin de semana, salvo en la semana de cierre de notas, cuando Lujan queda de guardia para S1.

Severidad	Definición	Respuesta	Resolución
Crítica (S1)	El sistema es inusable o hay pérdida o corrupción de calificaciones (ej. un promedio se guarda con un valor incorrecto).	4 horas hábiles	1 día hábil
Alta (S2)	Funcionalidad principal afectada sin alternativa y sin pérdida de datos (ej. Exportar PDF falla en materias de más de 50 estudiantes, CAL-001).	1 día hábil	2 días hábiles
Media (S3)	Funcionalidad secundaria afectada o existe alternativa (ej. el filtro por gestión falla, pero la materia se encuentra por nombre).	1 día hábil	1 semana

Severidad	Definición	Respuesta	Resolución
Baja (S4)	Defecto cosmético o mejora menor (ej. texto desalineado en el pie del PDF).	3 días hábiles	Siguiente sprint

9. Herramientas, técnicas y metodologías

Propósito	Herramienta	Versión	Práctica del curso
Control de versiones	Git + GitHub (GitHub Flow, main protegida)	Git ≥ 2.40	—
Integración continua	Jenkins (Jenkinsfile en la raíz del repositorio)	LTS	6
Análisis estático	SonarQube, Quality Gate "CalisFacu"	Community	3
Pruebas unitarias y de integración	PHPUnit + PCOV; coverage-check	11.x	1 y 2
Pruebas E2E	Cypress	13.x	4
Pruebas de carga	Apache JMeter	5.6	5
Análisis de dependencias	composer audit + Dependabot	—	3
Gestión de defectos	GitHub Issues	—	—

Técnicas: TDD obligatorio en Registro de calificación y Cálculo de promedios; revisión de código por pull request obligatoria en todo merge a main (Sección 6); análisis estático en cada pull request. El pair programming no se usa como técnica general por el tamaño del equipo, sino para transferir el conocimiento de los módulos críticos (mitigación del riesgo R01).

10. Control de medios

Pendiente: se desarrolla en la Parte C (Ejercicio 4). Para el SQAP mínimo rige lo indicado en las Secciones 6 y 9: rama main protegida y merge solo con un pull request aprobado y el pipeline en verde.

11. Control de proveedores

Pendiente: se desarrolla en la Parte C (Ejercicio 4). Para el SQAP mínimo, las dependencias se controlan con composer audit y Dependabot en cada pull request (M08, riesgo R06).

12. Registros: recolección, mantenimiento y retención

Evidencia	Ubicación	Formato	Retención	Responsable
Actas de revisión, inspección y auditoría	docs/actas/ en el repositorio	PDF firmado	Toda la vida del proyecto	Galvan (SQA)
Informes de pruebas (PHPUnit, Cypress)	Artefactos del build en Jenkins; los de cada release candidate, también en docs/evidencias/	JUnit XML y HTML	30 builds en Jenkins; los de release, toda la gestión	Galvan (SQA)
Dashboards de JMeter (M03, M04)	Artefactos del build de cada release candidate (perf/)	HTML + statistics.json	Toda la gestión	Galvan (SQA)
Registros del pipeline	Consola de Jenkins	Texto	Últimos 30 builds	Galvan (SQA)
Informes de SonarQube (M05, M07, M08)	Servidor SonarQube del equipo, proyecto CalisFacu	Web	6 meses	Galvan (SQA)
Reportes de defectos	GitHub Issues del repositorio	Issue con plantilla	Toda la vida del proyecto	Lujan (Líder)

13. Capacitación

Pendiente: se desarrolla en la Parte C (Ejercicio 3).

14. Gestión de riesgos

Escala de probabilidad (P) e impacto (I) de 1 (bajo) a 3 (alto); exposición = P × I; nivel: 1–2 bajo, 3–4 medio, 6–9 alto. El disparador y el plan de contingencia de cada riesgo están en la hoja Riesgos de matrices-sqap.xlsx. Todo riesgo alto tiene una mitigación que se puede poner como tarea en el tablero del equipo, con responsable y momento.

ID	Riesgo	P	I	Exp.	Mitigación accionable	Resp.
R01	Abandono o ausencia prolongada de un integrante (equipo de dos)	3	3	9 Alto	Cada historia se documenta en la wiki del repositorio antes de cerrarse; todo PR lo revisa el otro integrante; sesión de pair programming semanal sobre los módulos críticos.	Lujan
R02	Reglas de cálculo de promedios ambiguas (ponderaciones, redondeo)	2	3	6 Alto	Validar el reglamento de evaluación con el cliente en la semana 1; tabla de casos de prueba con resultados esperados firmada por el cliente antes del sprint 2.	Lujan
R03	Deuda técnica por presión de cronograma	2	3	6 Alto	Quality Gate bloqueante (M05, M07); 10 % de cada sprint reservado para refactorizar.	Galvan
R04	Falla del servidor de integración continua	1	2	2 Bajo	Jenkinsfile y docker-compose versionados; reinstalación probada en la semana 2.	Galvan
R05	Lentitud o caída en el cierre de notas por concurrencia (generación de PDF con DomPDF)	3	3	9 Alto	Generar el PDF en una cola de Laravel (Queue) para materias de más de 50 estudiantes, antes del RC2; prueba de carga en cada release candidate (M03, M04).	Lujan
R06	Vulnerabilidad crítica en una dependencia de Composer o npm	2	3	6 Alto	Dependabot activo y composer audit en cada PR (M08); revisión mensual de dependencias el primer lunes de cada mes.	Lujan
R07	Datos de prueba con calificaciones reales de estudiantes	1	3	3 Medio	Seeders y factories con datos sintéticos versionados; prohibido copiar la base real.	Galvan

15. Glosario

Término	Definición
CalisFacu	Sistema web de registro de calificaciones cubierto por este plan.
Módulo crítico	Módulo cuya falla afecta notas oficiales: Registro de calificación y Cálculo de promedios.

Término	Definición
Promedio ponderado	Nota final calculada con las ponderaciones de evaluación definidas por el docente (RNF-01).
p95	Valor por debajo del cual está el 95 % de los tiempos de respuesta medidos.
Release candidate (RC)	Versión candidata a entregarse, en una rama release/*, sobre la que se corren las etapas E2E y Rendimiento.
Staging	Ambiente de pruebas con la misma configuración que producción y datos sintéticos.
Quality Gate	Conjunto de condiciones de SonarQube que un cambio debe cumplir para aceptarse.
PCOV	Extensión de PHP que mide la cobertura de código de PHPUnit.
Feature test	Prueba de Laravel que ejercita una ruta HTTP completa, con base de datos.
Severidad	Cuánto daño causa un defecto (S1 a S4). La fija el Tester o el Responsable SQA.
Prioridad	Con qué urgencia se corrige un defecto. La fija el Líder.
SLA	Tiempos máximos de respuesta y resolución de un defecto según su severidad.
RACI	Matriz que asigna, por actividad, quién ejecuta (R), rinde cuentas (A), es consultado (C) o informado (I).
TDD	Desarrollo guiado por pruebas: se escribe la prueba antes que el código.

16. Procedimiento de cambio e historia del SQAP

Cualquier integrante propone un cambio con un pull request sobre docs/SQAP-CalisFacu.docx. Lo aprueba el Responsable SQA; si el cambio modifica un umbral, también lo aprueba el Líder, y en el mismo PR se actualizan matrices-sqap.xlsx y la configuración de la herramienta. Cada cambio es una fila de este historial.

Autoría por sección: Secciones 1 a 4, 6, 9 y 15: autor Lujan, revisor Galvan. Secciones 5, 7, 8, 12 y 14: autor Galvan, revisor Lujan.

Versión	Fecha	Cambio	Autor	Revisor
0.1	04/10/2026	Versión inicial: Secciones 1 a 8.	Lujan	Galvan

Versión	Fecha	Cambio	Autor	Revisor
0.2	04/10/2026	Corrección tras la revisión contra los siete indicadores: métricas M01 a M09 iguales en el plan, en matrices-sqap.xlsx y en la configuración; vulnerabilidades pasan a Seguridad; RACI con independencia; SLA en horario hábil; diagrama de estados; Secciones 9, 12, 14 y 15; Anexo de RNF.	Galvan	Lujan

Anexo — Extracto de requerimientos no funcionales (SRS 1.0)

Código	Requerimiento
RNF-01	El promedio final de cada estudiante se calcula con las ponderaciones de evaluación definidas por el docente para la materia, sobre 100 puntos y redondeado al entero más cercano; la nota mínima de aprobación es 51.
RNF-02	Un docente debe poder registrar las calificaciones de todos los estudiantes de una materia y verlas reflejadas en el promedio sin errores de guardado.
RNF-03	La exportación del historial de una materia a PDF debe responder en 2 segundos o menos para el 95 % de las solicitudes con 100 usuarios concurrentes (cierre de gestión).
RNF-04	Las calificaciones son datos personales y oficiales: solo el docente de la materia puede modificarlas y el sistema no debe tener vulnerabilidades conocidas.
RNF-05	El código debe poder ser mantenido por el equipo de la siguiente gestión.